SDG-03

Space Mission Operations Automator

Problem statement Build a simulation-first mission-operations system that plans activities, monitors telemetry, detects anomalies, schedules resources, and proposes procedure steps while keeping every real or irreversible command behind explicit authority and verification.

AT A GLANCE

Design dimension	Participant expectation
Primary user	Mission planners, flight controllers, and spacecraft-operations teams in a simulator.
Core inputs	Mission goals, timeline, telemetry, constraints, procedures, resources, and simulated faults.
Required outputs	Plan, constraint checks, anomaly diagnosis, procedure recommendation, and operator trace.
Hardest challenge	Coordinating time, resources, safety constraints, and partial observability without unsafe automation.
Proof of quality	Digital-twin replay, fault injection, constraint satisfaction, and operator approval workflow.

Core objective

Create an operations automator for a simulated satellite, rover, observatory, or interplanetary mission. The system should turn mission goals into a schedule, monitor telemetry, detect and characterize anomalies, propose safe procedure steps, manage communication windows and resources, and preserve the operator as the final authority for commands.

Required capabilities and system blueprint

1. Mission model or digital twin for spacecraft state, instruments, power, thermal, data storage, pointing, communications, and time-dependent constraints.
2. Planner that schedules observations, downlinks, maintenance, calibration, and safe-mode transitions while respecting resource and precedence constraints.
3. Telemetry pipeline with baseline modeling, limit checks, trend analysis, anomaly detection, alarm correlation, and uncertainty-aware diagnosis.
4. Procedure engine that retrieves versioned runbooks, checks preconditions, simulates effects, and proposes but does not silently execute commands.
5. Fault-injection harness for sensor drift, packet loss, power/thermal excursions, communication delay, conflicting telemetry, and actuator failure.
6. Operator console with timeline, state, evidence, approvals, command preview, rollback/abort semantics, and post-event review.
7. Mission memory for configuration, prior anomalies, procedures, and decisions with strict versioning and authority boundaries.

Minimum viable demonstration

8. Load a mission plan, schedule a constrained observation/downlink sequence, and show why the planner selected the order.
9. Inject a telemetry anomaly and show detection, competing hypotheses, evidence, and a recommended safe procedure.
10. Require operator approval for a command and show that the simulator previews and validates it before execution.
11. Replay the event and compare automated recommendation, operator decision, and simulated mission outcome.

Evaluation and benchmarks

12. Use an open mission simulator, synthetic telemetry, or organizer-provided digital-twin tasks; document assumptions and units.
13. Measure schedule feasibility, resource violations, anomaly precision/recall, time to detection, diagnosis ranking, safe-procedure success, and operator workload.
14. Include fault-injection tests, unseen combinations of faults, delayed observations, and recovery from a failed plan.

Hard-mode extensions

15. Conflicting sensors, missing telemetry, long communication delays, multiple simultaneous faults, changing mission priorities, and limited onboard resources.
16. No system should transmit real commands or interface with a live spacecraft. The challenge is deliberately simulation-first and approval-gated.

> **Safety boundary** Keep the entire system in a simulator or approved lab. Never connect an autonomous agent to live spacecraft, flight hardware, or mission credentials. Command generation must be separated from command authorization, and every simulated action must be replayable.

Expected deliverables

17. Mission digital twin or simulator with telemetry generator and fault injection.
18. Planner, anomaly detector, procedure/retrieval engine, operator console, and trace store.
19. Constraint, safety, authority, and versioning model.
20. Replayable evaluation suite and fault-analysis report.
21. Demo showing an anomaly, approval-gated procedure, and recovery outcome.

Why this stands out

Space operations are a compelling test of agentic reliability because the system must reason over time, constraints, partial observability, and consequences. A simulator makes the challenge ambitious while keeping experimentation safe and repeatable.